TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

TECHNICAL REPORT

Field	Details
Institute / University	Pimpri Chinchwad College of Engineering (PCCOE)
Programme / Batch	B.Tech Computer Engineering | Tech Pulse FY-26
Student Name	Atharv Rao
PRN	123B1B024
Case Study ID (CS1–CS5)	CS1 – AUTOSAR HLD Document Analysis Assistant
Project Title	AUTOSAR Architecture Intelligence Assistant

1. Problem Statement and Intended Users

AUTOSAR High-Level Design (HLD) documents spread software components, interfaces, ports, signals, dependencies and revision history across narrative text, tables and document structure. Engineers need fast answers that stay traceable to the source document and that never treat a language-model response as approval authority. The AUTOSAR Architecture Intelligence Assistant turns HLD documents into an evidence-linked, human-validated architecture knowledge graph stored in PostgreSQL. Engineers can ask natural-language questions (answers carry citations, confidence, extraction type and validation status), explore the graph, analyse the impact of a change, compare document revisions, review and correct extracted facts and export structured results. Intended users are system architects, AUTOSAR and software engineers, integration engineers and reviewers. The system assists engineers; it does not approve an architecture.

2. Input Data / Knowledge Base

Item	Description
Sources	Three synthetic AUTOSAR-style HLD fixtures written for this project: BrakeController HLD v1 and its revision v2, and a separate ABS subsystem HLD (interfaces, dependencies, a timing section and a diagnostics table of data identifiers, declared status 'Approved'). A labelled set of 23 questions for the ABS HLD is used for the retrieval evaluation. No confidential or customer material is included.
Size and format	UTF-8 Markdown, about 1-2 KB each (7, 5 and 8 sections). The pipeline also accepts PDF, DOCX, XLSX and CSV (50 MB upload limit); v2 is uploaded under the same file name as v1 so that it is recorded as version 2.
Preprocessing / EDA done	Extension, MIME and size validation; SHA-256 content hash (identical bytes are rejected); managed storage; header/footer cleaning; section tree with numbering and levels; table rows kept as rows; declared metadata (version, classification, status) read from the document; OCR for scanned pages behind a provider boundary (local Tesseract or consent-gated Gemini vision) with a page cache. OCR was not exercised in the evaluation.
Chunking strategy	Structure-aware chunks (target 400, maximum 800 tokens; 80-token overlap only when a section has to be split). Every chunk is embedded together with its breadcrumb (document > section path) so a child keeps its parent context; retrieval adds the parent lead and neighbouring chunks.

3. Solution Architecture

Component	Choice (name, version, hosting)
Pre-trained model	Groq OpenAI-compatible API, openai/gpt-oss-120b (hosted, external). Used for candidate extraction, ambiguous entity-merge decisions and answer drafting; reached only through LLMProvider.
Embedding model	Gemini native embeddings, gemini-embedding-2, 768 dimensions (hosted, external), reached only through EmbeddingProvider. Model name and version are stored per chunk; changing them requires an explicit re-embed.
Vector store (FAISS / ChromaDB)	PostgreSQL 18 + pgvector (vector(768), HNSW cosine index) as the system of record, plus full-text search (tsvector + GIN) for the lexical channel. The graph is relational: entities, relationships and evidence tables, project-scoped. No FAISS, ChromaDB or Neo4j.

Component	Choice (name, version, hosting)
RAG pipeline / AI-agent design	Dense + lexical retrieval fused by reciprocal-rank fusion with a relevance gate; parent and neighbour context; graph facts from a bounded PostgreSQL recursive CTE; evidence bundle; sufficiency check (at most two rounds); LLM drafts claims that must cite chunk ids; every claim is verified against its cited text; unsupported claims are dropped; otherwise 'Insufficient evidence'.
Service layer and UI	React 18 + TypeScript + Vite with a 3D graph (three.js); FastAPI + Pydantic v2; SQLAlchemy 2 + Alembic (12 migrations); PostgreSQL job queue with a separate worker (SKIP LOCKED leases, checkpoints, retries); Docker Compose for deployment.
Tools / deterministic rules	Pydantic validation; controlled relationship taxonomy; entity resolution (blocking, guards, rules, constrained Union-Find; LLM only for ambiguous pairs); confidence as a pure function; persisted findings worded 'Potential inconsistency detected'; impact-propagation rules; project authorization; audit log; secret-scan script.

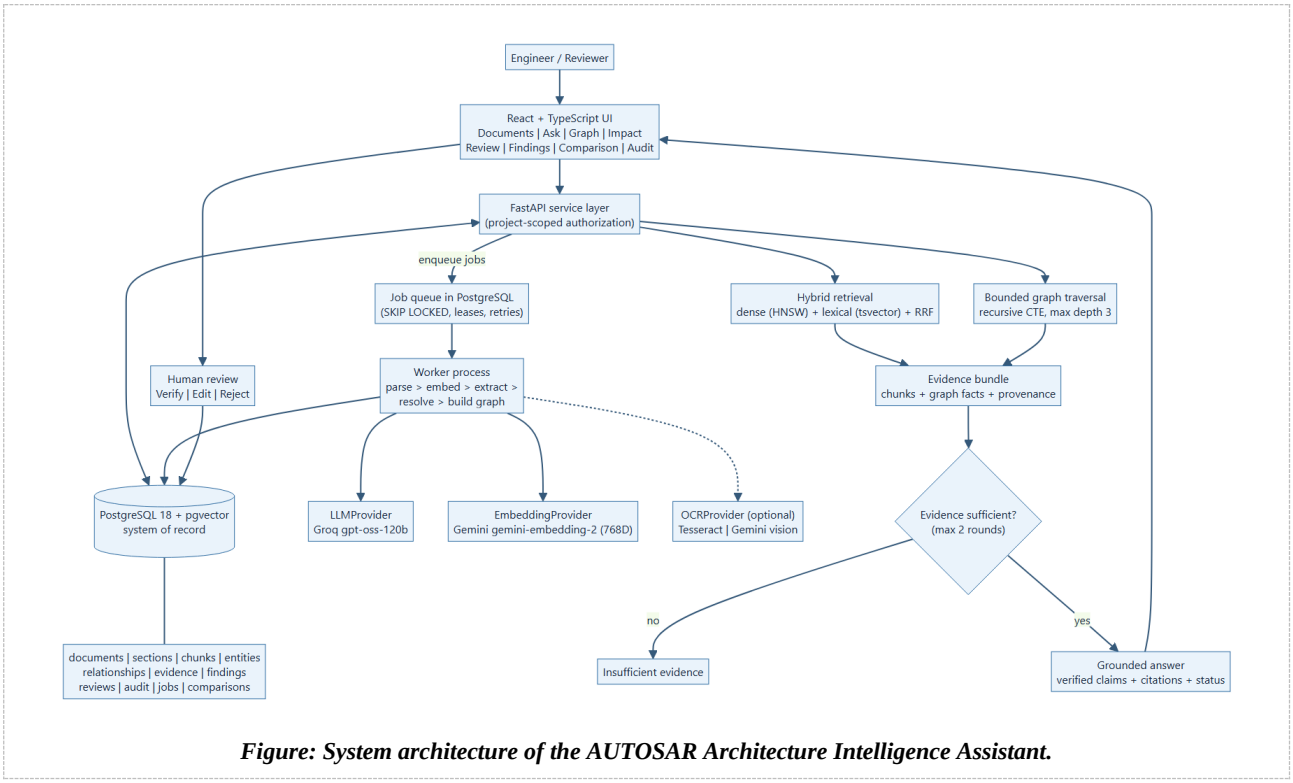


Figure: System architecture of the AUTOSAR Architecture Intelligence Assistant.

4. Implementation and Configuration

Pipeline. A document enters a one-at-a-time upload queue and is validated (type, size, SHA-256; identical bytes are rejected). A PostgreSQL-backed worker then runs five explicit, checkpointed and retryable jobs: DOCUMENT_INGESTION (parse, clean, section tree, structure-aware chunks, OCR if needed), EMBEDDING (breadcrumb + content to pgvector), GRAPH_EXTRACTION (the LLM proposes candidate entities and staged relationships; Pydantic validation; controlled taxonomy), ENTITY_RESOLUTION (blocking, guards, rules, LLM only for ambiguous pairs, constrained Union-Find; soft, undoable merges) and GRAPH_BUILD (bind relationships, validate, pure confidence function, persisted findings). The interface shows the five stages and ticks each one as its job completes. Everything the pipeline creates is PENDING.

Query. A question is linked to entities and planned (intent, direction, depth). Dense (pgvector HNSW) and lexical (tsvector) retrieval are fused by reciprocal-rank fusion behind a relevance gate, parent and neighbouring context is added, and PostgreSQL traverses the graph with a bounded recursive CTE (depth up to 3). The evidence bundle is checked for sufficiency (two rounds at most); the LLM drafts claims that must cite chunk identifiers, each claim is verified against its cited text and unsupported claims are dropped. If nothing survives, the answer is 'Insufficient evidence'.

Configuration. Python 3.11+ (developed on 3.14), FastAPI, SQLAlchemy 2, Alembic (12 migrations), PostgreSQL 18 with pgvector and pg_trgm; React 18, TypeScript, Vite and react-force-graph-3d; Groq gpt-oss-120b and Gemini gemini-embedding-2 (768 D) behind provider interfaces. Defaults: chunks of 400/800 tokens, top-k 8, two retrieval rounds, claim support of at least 0.6. The extraction prompt is versioned (autosar-candidate-extraction-v4). Settings, prompts and run instructions are in Model_Prompts_Config/ and Documentation/RUN_AND_DEPLOY.md; no credentials are included.

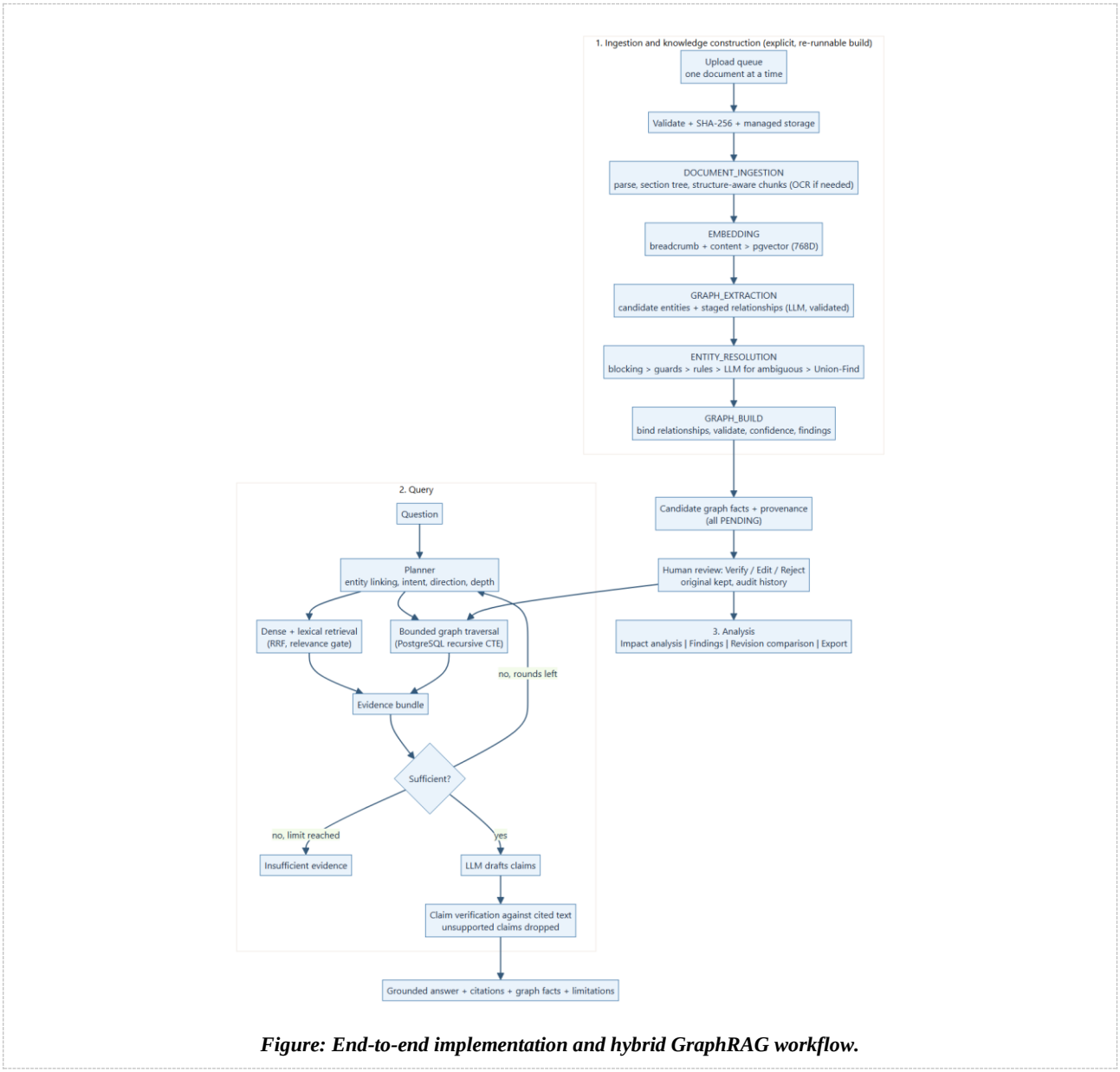


Figure: End-to-end implementation and hybrid GraphRAG workflow.

5. Evaluation Steps and Results

Two kinds of evidence were produced on 14 October 2026, both on synthetic documents only. (1) A retrieval and answer evaluation: 23 labelled questions on the ABS HLD (19 answerable, 4 unanswerable) run through the real ingestion and answer services in a throw-away PostgreSQL database, once offline (deterministic stand-in models) and once with the live providers; the ground truth and manual scoring rules are in Evaluation_Results/. (2) An end-to-end run through the real UI and worker with live providers, reviewed by hand. The earlier synthetic '9/9' harness was removed because it constructed part of its evidence in memory. The numbers are small-sample indicators, not accuracy estimates.

#	Test question / scenario	Expected answer / behaviour	Result (Pass/Fail, notes)
1	Live retrieval evaluation: 23 questions on the ABS HLD (components, aliases, interfaces, dependencies, ports, identifiers, timing, diagnostics, hierarchy, 4 unanswerable).	Expected section retrieved; citations valid; unanswerable questions refused.	PASS with limits - Recall@8 100%, MRR 0.97, citation validity 100%, refusal accuracy 4/4. FAIL on 2 of 19 answerable questions (10.5% false refusals; claim verification withdrew an alias answer and a 'where is it described' answer).
2	'What does BrakeController require?' (Hybrid).	BrakeController REQUIRES WheelSpeedInterface, cited.	PASS - one claim at 100% support, two citations, graph fact shown as Confidence 1.00 / EXPLICIT / PENDING.
3	'Which DID corresponds to the vehicle speed signal and what is its data length?' (Source only).	0xF40D = VehicleSpeed, length 1 (table row).	PASS - DID correct, cited to section 3.2 (Approved); the length claim was verified at the 0.6 threshold.
4	'What is the maximum supported braking torque ... in newton-metres?' (fact absent from all documents).	Refuse instead of inventing a value.	PASS - 'Insufficient evidence', groundedness 0%.
5	Human review: verify, reject and edit extracted facts through the UI.	Decision recorded with a reason; original kept; status shown by colour and text.	PASS - 2 verified, 1 rejected, 1 corrected; the corrected original is stored as superseded; all events in validation_events.
6	Impact analysis of WheelSpeedInterface, 2 hops.	Direct and indirect elements with the connecting facts.	PASS - 4 direct and 7 indirect potentially affected elements, each with path and weakest-link status. Whether each is a real impact is an engineering judgement.
7	Revision comparison BrakeController HLD v1 -> v2.	Added, removed, renamed and changed elements with sources from both versions.	PASS with caveat - 90 findings (56 removed, 18 added, 10 modified, 4 target changed, 2 renamed); BrakeCommandInterface flagged as possibly renamed. Removals are noisy for a heavily rewritten document.
8	Automated tests and static checks.	All pass.	PASS - 392 backend tests (1 live-provider test skipped), 29 frontend tests; ruff, strict mypy, eslint, tsc and the production build are clean.

Metric	Score / observation
Retrieval (live, 23 questions)	Recall@8 100%; MRR 0.974; context recall 100%. Offline stand-in: 94.7% / 0.895.
Citation validity / refusals	Citation validity 100%; refusal accuracy on unanswerable questions 100% (4 of 4), offline and live.
Conservatism cost	False refusals 10.5% (2 of 19) live, 5.3% offline; 13.6% of generated claims dropped as unsupported.
Latency (live providers)	Whole answer mean 13.7 s, p95 26.4 s (refusals and two-round answers 20-30 s). Pipeline: extraction 31-75 s per document, other stages 0.0-13 s.
Knowledge graph from the run	35 live entities and 24 live relationships, every relationship with at least one evidence row; 3 findings; extraction precision/recall against a labelled graph was NOT measured.

Failures and limits. Two answerable questions were refused although the right section was retrieved at rank 1; the model's statement could not be supported from the cited text (an alias-only paraphrase, and a statement about where something is described). The extractor proposed

generic entities (ABS, ESP as functions) that a reviewer rejected, and needed a prompt revision (v4) after producing reversed CARRIES edges. The question set is small, written by the student and not independently reviewed; it covers one short document, no scanned pages and one pair of models.

6. Responsible AI Measures

Area	Measure adopted in this project
Grounding and citations	Every graph fact links to evidence (relationship -> evidence -> chunk -> section -> document version). Answer claims must cite retrieved passages and are verified against them; unsupported claims are dropped; absent facts are refused (4 of 4 in the evaluation). Explicit and inferred facts are distinguished.
Transparency of uncertainty	Confidence is never shown alone: it is paired with extraction type and validation status. Groundedness is labelled as a grounding indicator, not a probability of correctness. Findings say 'Potential inconsistency detected', never 'the architecture is wrong'.
Privacy	Only synthetic documents are used with hosted models. Local/private inference (Ollama) is the target for confidential documents. External OCR needs an explicit consent flag (off by default). Secrets stay in local .env files and are excluded from the package; a scanner checks folders and archives.
Security	Project-scoped access and isolation, upload validation, managed storage, path safety, prompt-injection separation (documents are data, not instructions), no model-written SQL, no direct LLM writes to trusted state, audit log, safe error handling.
Human oversight / review	Every extracted fact starts PENDING; the pipeline never sets HUMAN_VERIFIED. Reviewers verify, edit or reject with a recorded reason; corrections keep the original and the full history. Entity merges are soft and undoable.
Reproducibility	Graph construction is an explicit, idempotent build, not a side effect of a question. Prompts and embedding models are versioned; confidence is a pure function of stored inputs, so rebuilds do not inflate it.

7. Innovation Highlights

Beyond the baseline case study, the project adds the following (all implemented and tested):

- Provenance-first relational knowledge graph in PostgreSQL: every entity and relationship is tied to source evidence, document version, section/chunk, confidence, extraction type and validation state (no separate graph database).
- Hybrid GraphRAG: dense (pgvector HNSW) and lexical (tsvector) retrieval fused by RRF, plus deterministic bounded graph traversal in a recursive CTE, an evidence-sufficiency loop capped at two rounds and per-claim verification against the cited text.
- Structure-aware ingestion: section tree, table rows kept as rows, breadcrumb embedded with every chunk, optional OCR behind a consent-gated provider boundary.
- Entity resolution pipeline (blocking, guards, rules, LLM only for ambiguous pairs, constrained Union-Find) with soft, undoable merges that never rely on a hash as proof of identity.
- Human-governed workflow: confidence is separate from validation state; verify / edit / reject keep the original candidate and an audit trail; nothing is auto-verified.
- Engineering analyses on the same graph: change-impact analysis with propagation rules, persisted 'potential inconsistency' findings that stay dismissed, revision comparison (document, section, entity and relationship level) and JSON/CSV/Excel export.
- Operator experience: a one-at-a-time upload queue with a five-stage tracker that ticks each stage as its job completes, a 3D graph that shows review state by colour and text, and a styled, keyboard-accessible selector.

8. Screenshots, Logs and Evidence

Evidence was captured on 14 October 2026 from the running application with live providers and synthetic data. The full set is in Evaluation_Results/Screenshots/ (28 screens), granular_screenshots/ (19 element crops) and psql/ (16 database views, drawn from real psql output; see EVIDENCE_INDEX.md). The figures below show the main workflow and the stored data.

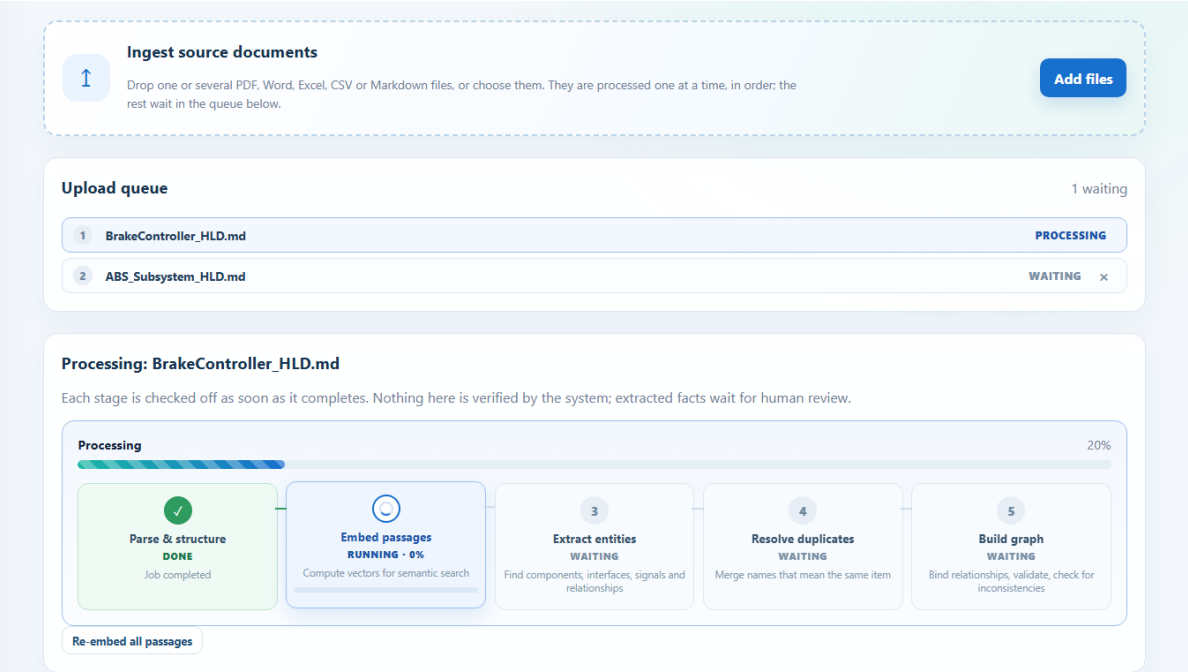


Figure 1. Sequential upload queue and five-stage pipeline tracker: Parse & structure done, Embed passages running, the rest waiting.

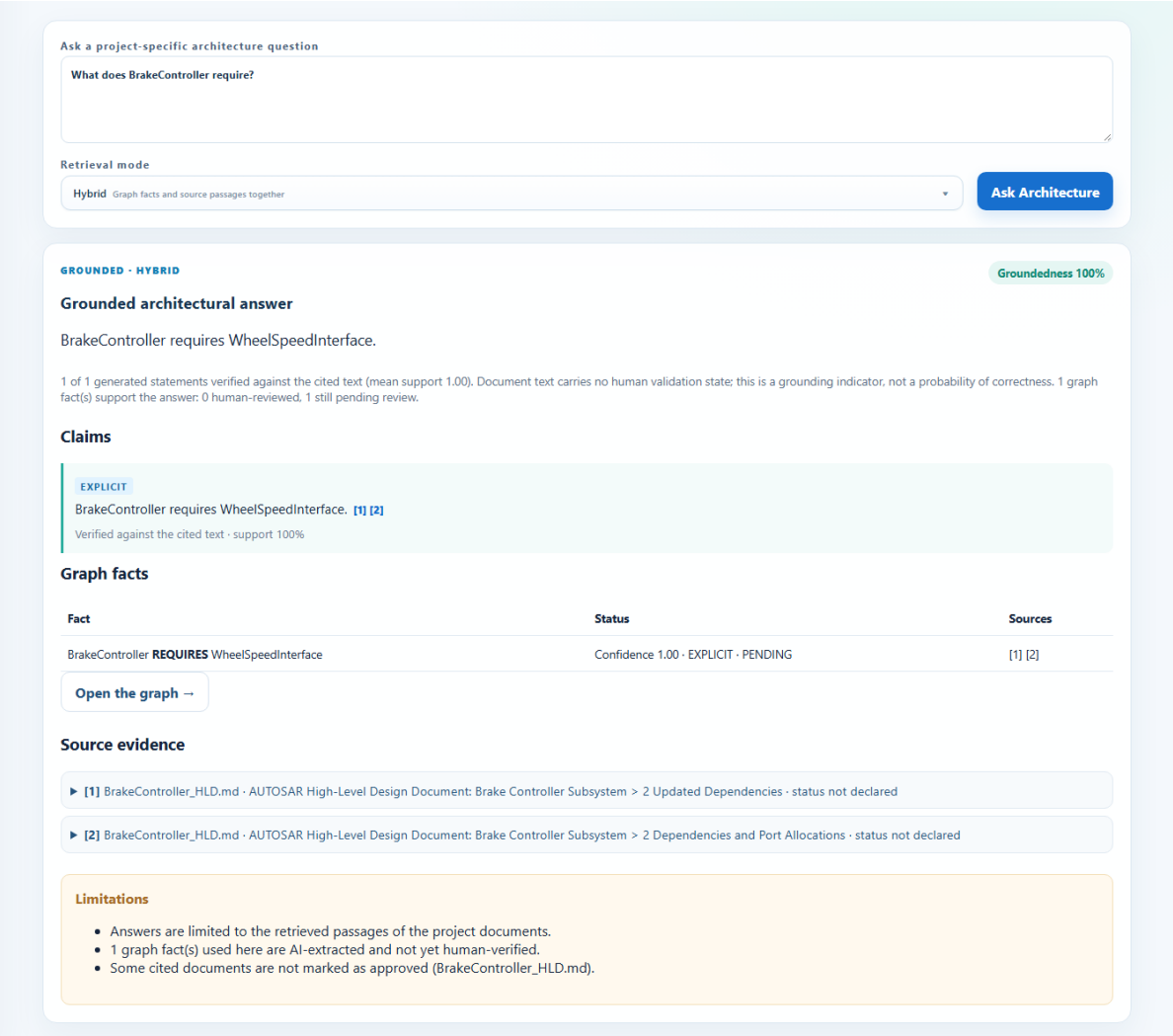


Figure 2. Grounded hybrid answer: claim with citation markers, graph fact with Confidence / Extraction / Validation, source evidence and limitations.

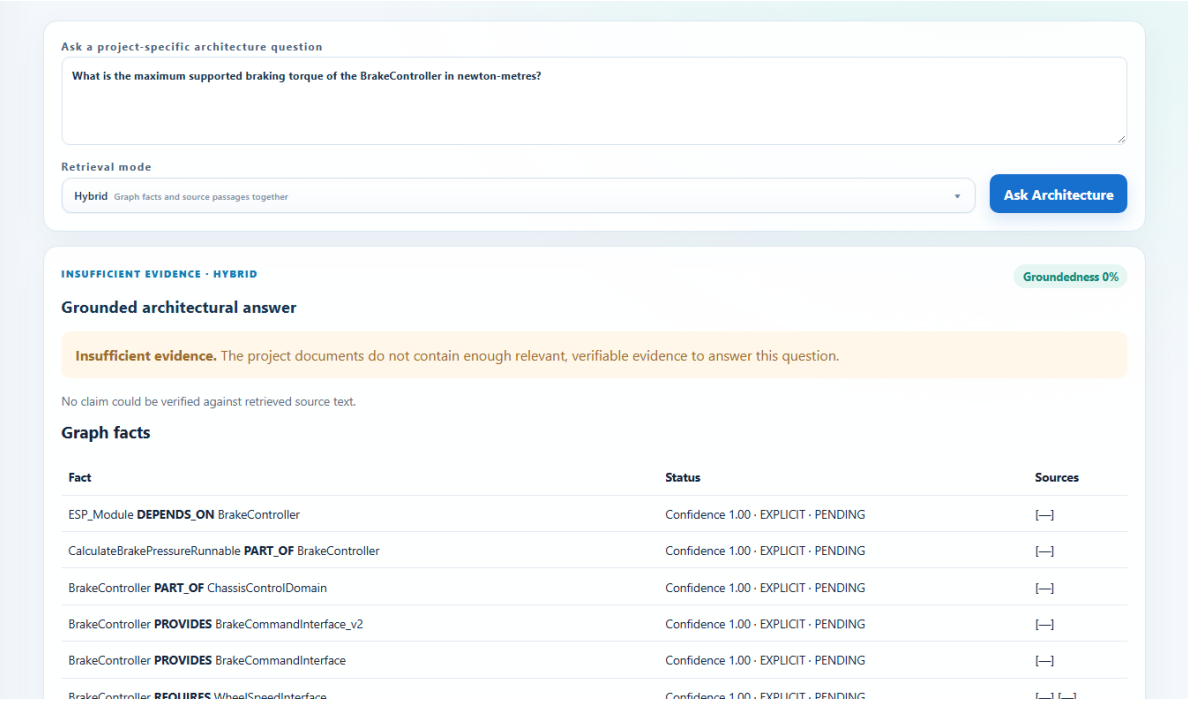


Figure 3. A fact that is absent from every document (braking torque) is refused as 'Insufficient evidence'.

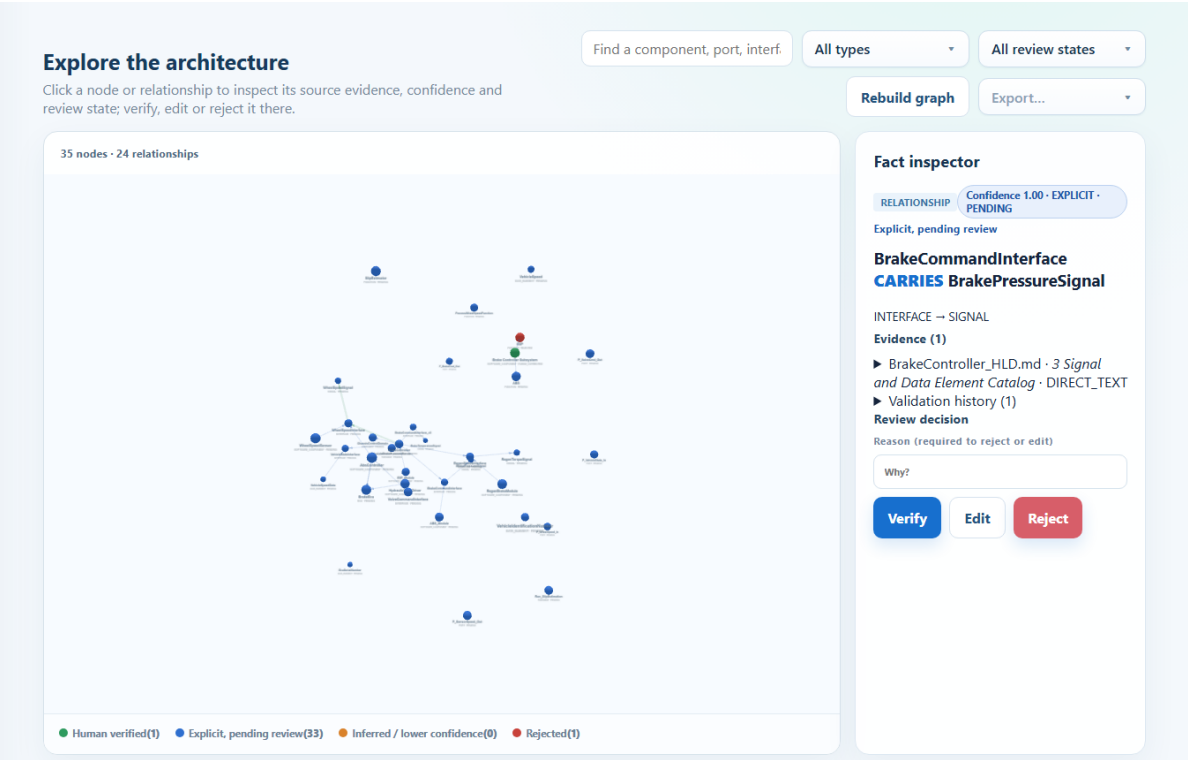


Figure 4. Knowledge graph (35 nodes, 24 relationships) with the fact inspector for one relationship; colours and text show review state.

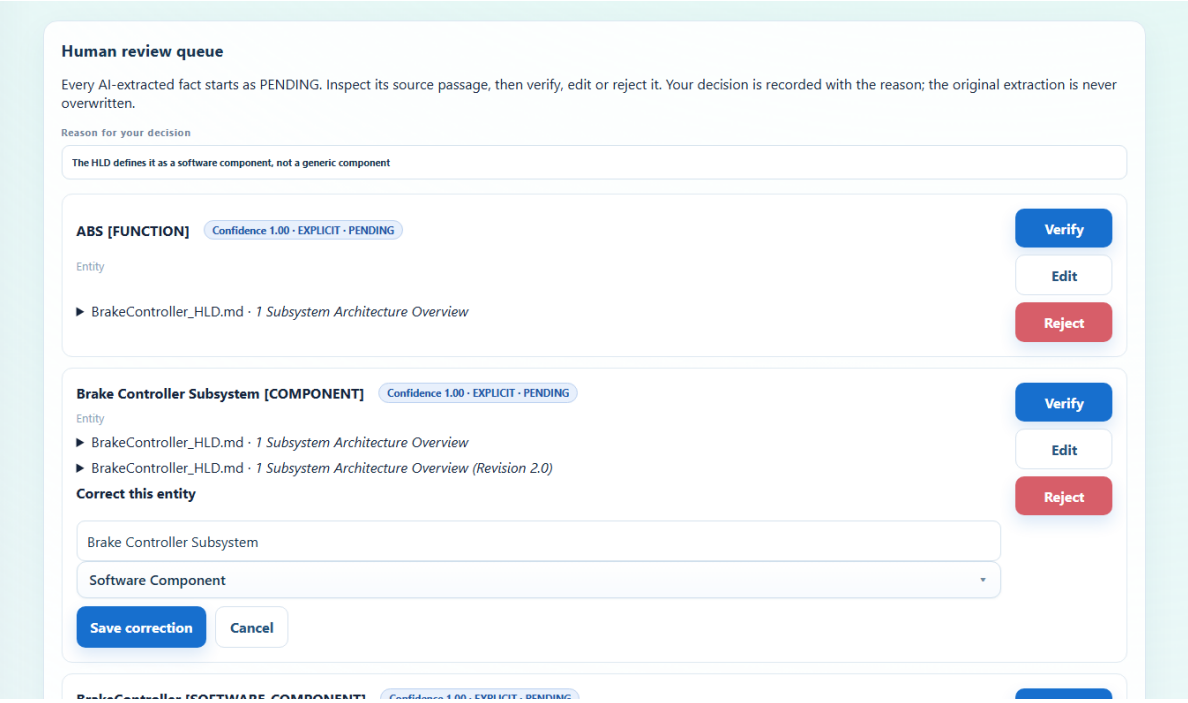


Figure 5. Human review queue with source passages and the edit form; the original extraction is kept in the history.

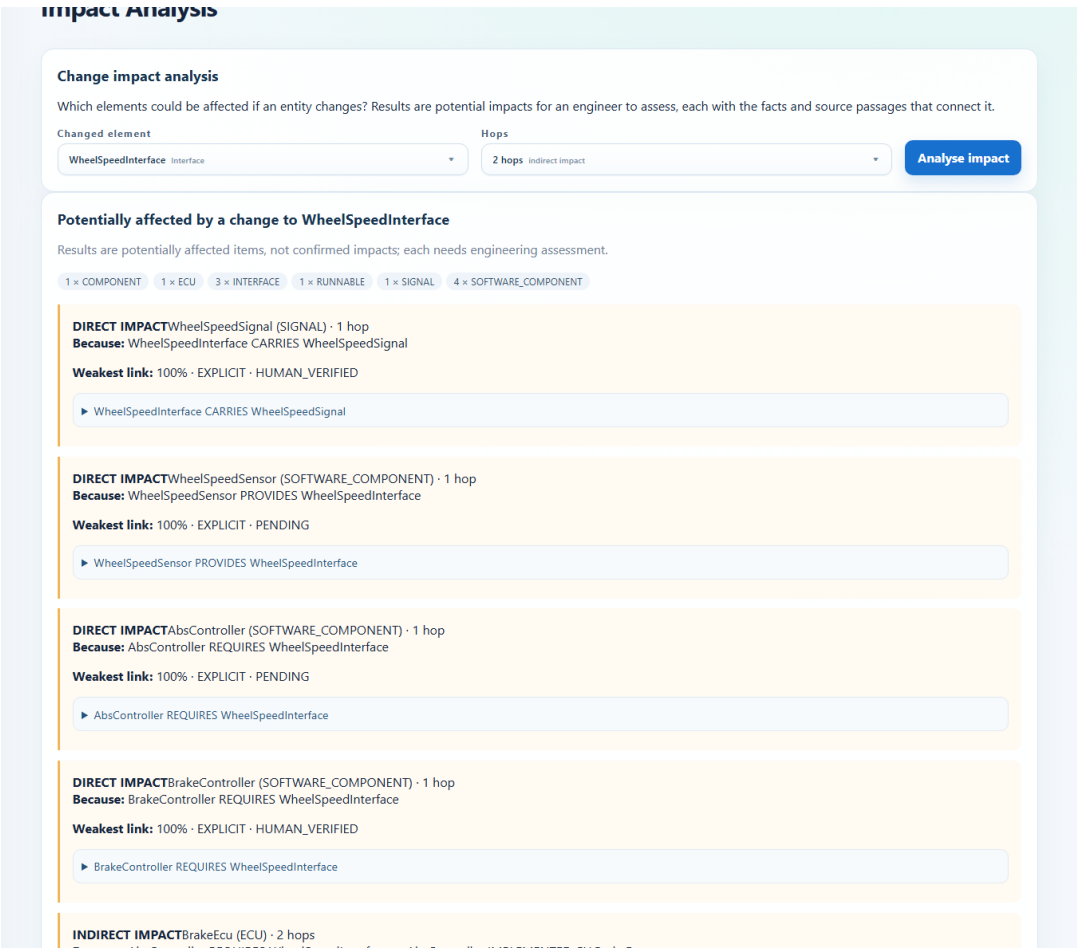


Figure 6. Impact analysis of WheelSpeedInterface (2 hops): direct and indirect potentially affected elements with their paths.

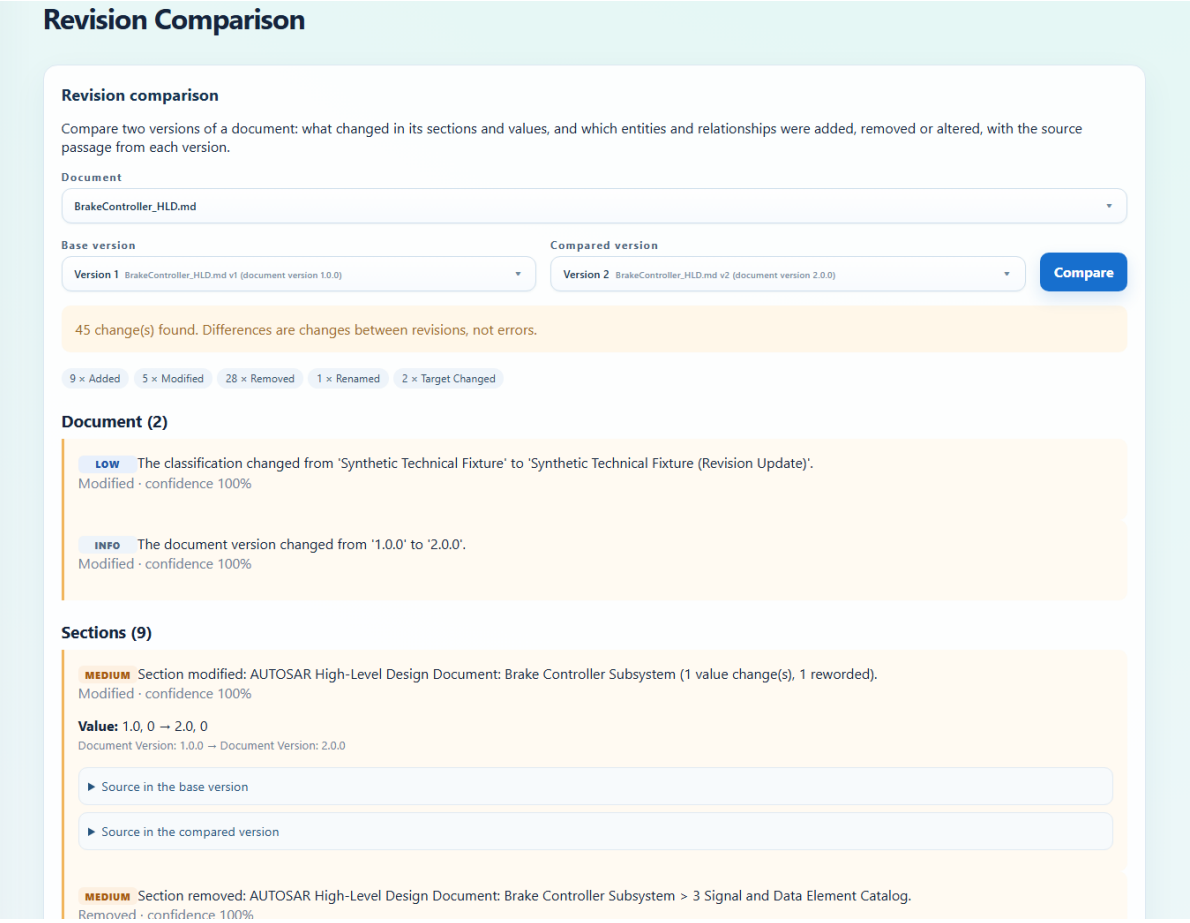


Figure 7. Revision comparison of BrakeController HLD version 1 and version 2: document, section, entity and relationship changes.

```
psql (PostgreSQL 18)  -  database: autosar_evidence  -  host: localhost

autosar_evidence=# \d chunks
                                                    Table "public.chunks"
        Column          |           Type           | Collation | Nullable |                                                  Default
------------------------+--------------------------+-----------+----------+-----------------------------------------------------------------------------------------------------------
 project_id             | uuid                     |           | not null |
 document_version_id    | uuid                     |           | not null |
 section_id             | uuid                     |           | not null |
 ordinal                | integer                  |           | not null |
 content                | text                     |           | not null |
 token_count            | integer                  |           | not null |
 page_start             | integer                  |           |          |
 page_end               | integer                  |           |          |
 embedding              | vector(768)              |           |          |
 id                     | uuid                     |           | not null |
 created_at             | timestamp with time zone |           | not null | now()
 updated_at             | timestamp with time zone |           | not null | now()
 embedding_model        | character varying(255)   |           |          |
 embedding_model_version | character varying(128)   |           |          |
 embedded_at            | timestamp with time zone |           |          |
 extracted_at           | timestamp with time zone |           |          |
 breadcrumb             | text                     |           |          |
 content_hash           | character varying(64)    |           |          |
 chunk_kind             | character varying(16)    |           | not null | 'TEXT'::character varying
 attrs                  | jsonb                    |           | not null | '{}'::jsonb
 lexical_text           | text                     |           |          |
 search_vector          | tsvector                 |           |          | generated always as (to_tsvector('english'::regconfig, COALESCE(lexical_text, ''::text))) stored
Indexes:
    "pk_chunks" PRIMARY KEY, btree (id)
    "ix_chunks_content_hash" btree (content_hash)
    "ix_chunks_document_version_id" btree (document_version_id)
    "ix_chunks_embedding_hnsw" hnsw (embedding vector_cosine_ops)
    "ix_chunks_embedding_identity" btree (project_id, document_version_id, embedding_model, embedding_model_version)
    "ix_chunks_project_document_version" btree (project_id, document_version_id)
    "ix_chunks_project_id" btree (project_id)
    "ix_chunks_search_vector" gin (search_vector)
    "ix_chunks_section_id" btree (section_id)
    "section_chunk_ordinal" UNIQUE CONSTRAINT, btree (section_id, ordinal)
Foreign-key constraints:
    "fk_chunks_document_version_id_document_versions" FOREIGN KEY (document_version_id) REFERENCES document_versions(id) ON DELETE CASCADE
    "fk_chunks_project_id_projects" FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE
    "fk_chunks_section_id_sections" FOREIGN KEY (section_id) REFERENCES sections(id) ON DELETE CASCADE
Referenced by:
    TABLE "entity_evidence" CONSTRAINT "fk_entity_evidence_chunk_id_chunks" FOREIGN KEY (chunk_id) REFERENCES chunks(id) ON DELETE CASCADE
    TABLE "relationship_candidates" CONSTRAINT "fk_relationship_candidates_chunk_id_chunks" FOREIGN KEY (chunk_id) REFERENCES chunks(id) ON DELETE CASCADE
    TABLE "relationship_evidence" CONSTRAINT "fk_relationship_evidence_chunk_id_chunks" FOREIGN KEY (chunk_id) REFERENCES chunks(id) ON DELETE CASCADE
```

Figure 8. PostgreSQL: chunks table with vector(768), generated tsvector, HNSW and GIN indexes (psql \d chunks).

```
psql (PostgreSQL 18)  -  database: autosar_evidence  -  host: localhost

autosar_evidence=# SELECT s.canonical_name AS source, r.relationship_type AS rel, t.canonical_name AS target,
autosar_evidence-#        r.extraction_type AS extraction, r.validation_state AS validation, round(r.confidence, 2) AS conf,
autosar_evidence-#        d.name AS document, sec.number_label AS sect, left(re.excerpt, 44) AS excerpt
autosar_evidence-# FROM relationships r
autosar_evidence-# JOIN entities s ON s.id = r.source_entity_id
autosar_evidence-# JOIN entities t ON t.id = r.target_entity_id
autosar_evidence-# JOIN relationship_evidence re ON re.relationship_id = r.id
autosar_evidence-# JOIN chunks c ON c.id = re.chunk_id
autosar_evidence-# JOIN sections sec ON sec.id = c.section_id
autosar_evidence-# JOIN document_versions v ON v.id = c.document_version_id
autosar_evidence-# JOIN documents d ON d.id = v.document_id
autosar_evidence-# WHERE s.canonical_name = 'BrakeController' AND v.version_number = 1
autosar_evidence-# ORDER BY r.relationship_type, t.canonical_name;
     source      |   rel    |        target         | extraction |   validation   | conf |        document        | sect |                   excerpt
-----------------+----------+-----------------------+------------+----------------+------+------------------------+------+--------------------------------------------
 BrakeController | PART_OF  | ChassisControlDomain  | EXPLICIT   | PENDING        | 1.00 | BrakeController_HLD.md | 5    | `BrakeController` `PART_OF` `ChassisControlD
 BrakeController | PROVIDES | BrakeCommandInterface | EXPLICIT   | PENDING        | 1.00 | BrakeController_HLD.md | 2    | `BrakeController` `PROVIDES` `BrakeCommandIn
 BrakeController | REQUIRES | WheelSpeedInterface   | EXPLICIT   | HUMAN_VERIFIED | 1.00 | BrakeController_HLD.md | 2    | `BrakeController` `REQUIRES` `WheelSpeedInte
 BrakeController | USES     | VehicleStateInterface | EXPLICIT   | PENDING        | 1.00 | BrakeController_HLD.md | 2    | `BrakeController` `USES` `VehicleStateInterf
(4 rows)
```

Figure 9. PostgreSQL: provenance join from relationship through evidence, chunk, section and document version (psql).

```
psql (PostgreSQL 18)  -  database: autosar_evidence  -  host: localhost

autosar_evidence=# WITH RECURSIVE root AS (SELECT id FROM entities WHERE canonical_name = 'WheelSpeedInterface' AND merged_into_id IS NULL),
autosar_evidence-# live AS (
autosar_evidence-#     SELECT r.id, r.source_entity_id AS src, r.target_entity_id AS dst, r.relationship_type AS kind
autosar_evidence-#     FROM relationships r
autosar_evidence-#     JOIN entities s ON s.id = r.source_entity_id JOIN entities t ON t.id = r.target_entity_id
autosar_evidence-#     WHERE r.superseded_by_id IS NULL AND NOT jsonb_exists(r.attributes, 'inactive_reason')
autosar_evidence-#       AND s.merged_into_id IS NULL AND t.merged_into_id IS NULL
autosar_evidence-#       AND r.validation_state <> 'REJECTED' AND s.validation_state <> 'REJECTED' AND t.validation_state <> 'REJECTED'),
autosar_evidence-# walk (rel_id, src, dst, kind, to_id, depth, edge_path) AS (
autosar_evidence-#     SELECT l.id, l.src, l.dst, l.kind, CASE WHEN l.src = root.id THEN l.dst ELSE l.src END, 1, ARRAY[l.id]
autosar_evidence-#     FROM live l, root WHERE l.src = root.id OR l.dst = root.id
autosar_evidence-#   UNION ALL
autosar_evidence-#     SELECT l.id, l.src, l.dst, l.kind, CASE WHEN l.src = w.to_id THEN l.dst ELSE l.src END, w.depth + 1, w.edge_path || l.id
autosar_evidence-#     FROM walk w JOIN live l ON (l.src = w.to_id OR l.dst = w.to_id)
autosar_evidence-#     WHERE w.depth < 2 AND NOT (l.id = ANY (w.edge_path)))
autosar_evidence-# SELECT DISTINCT ON (w.rel_id) w.depth, s.canonical_name AS source, w.kind, t.canonical_name AS target
autosar_evidence-# FROM walk w JOIN entities s ON s.id = w.src JOIN entities t ON t.id = w.dst
autosar_evidence-# ORDER BY w.rel_id, w.depth;
 depth |             source             |      kind      |          target
-------+--------------------------------+----------------+-------------------------
     2 | BrakeController                | USES           | VehicleStateInterface
     1 | WheelSpeedSensor               | PROVIDES       | WheelSpeedInterface
     2 | AbsController                  | PART_OF        | BrakeEcu
     2 | CalculateBrakePressureRunnable | PART_OF        | BrakeController
     2 | AbsController                  | PROVIDES       | ValveCommandInterface
     1 | AbsController                  | REQUIRES       | WheelSpeedInterface
     2 | AbsController                  | IMPLEMENTED_BY | BrakeEcu
     2 | BrakeController                | PROVIDES       | BrakeCommandInterface_v2
     2 | BrakeController                | USES           | RegenStatusInterface
     2 | ESP_Module                     | DEPENDS_ON     | BrakeController
     2 | BrakeController                | PROVIDES       | BrakeCommandInterface
     2 | BrakeController                | PART_OF        | ChassisControlDomain
     1 | BrakeController                | REQUIRES       | WheelSpeedInterface
     1 | WheelSpeedInterface            | CARRIES        | WheelSpeedSignal
(14 rows)
```

Figure 10. PostgreSQL: bounded graph traversal (recursive CTE, depth 2) around WheelSpeedInterface (psql).

9. Reflection and Conclusions

What I learned. An engineering AI system must treat traceability as a requirement, not as a feature added afterwards: PostgreSQL with pgvector stayed the single system of record, the graph stayed relational and evidence-linked, and deterministic code does everything that does not need a model (parsing, resolution rules, traversal, confidence, claim verification). Measuring the system on the real pipeline was more valuable than the earlier synthetic harness: it exposed defects only a live model produces, for example models citing bare UUIDs instead of 'chunk:' identifiers (every such answer was dropped), correct table-row paraphrases failing claim verification, reversed CARRIES edges and generic-phrase entities in extraction. Each became a code or prompt fix with a regression test.

Challenges. A worker process and the API resolving a relative upload folder differently; numeric fields arriving as JSON strings and crashing the review screen; a 3D canvas sized to the window instead of its card; provider latency (20-30 s for refusals); and keeping a hosted-API dependency honest and replaceable behind provider interfaces.

Limitations. The evaluation set is small, synthetic and student-labelled; extraction precision and recall were not measured; OCR, Docker Compose and large multi-hundred-page HLDs were not tested; the question planner is rule-based; the claim verifier is conservative (2 of 19 answerable questions were refused); revision comparison is noisy when a document is rewritten; production identity (enterprise authentication) is an integration boundary, not implemented.

Future work. A larger, independently reviewed evaluation set including real (approved) HLDs and scanned pages; tuning of the relevance gate and claim threshold with the live evaluation; an LLM planner behind the same typed plan; local/private models for confidential documents; enterprise authentication and document-retention policy; and a native graph database only if multi-hop traversal becomes a demonstrated bottleneck.

10. AI-Tool Usage Declaration

AI tool / model	Purpose	Artifact affected	Verified by Student (Y/N)
Codex (OpenAI)	Development assistance under student direction in the first phase: implementation, debugging, tests and documentation.	Initial code, tests, configuration/docs, report/synopsis and submission artifacts.	Y
Claude Code (Anthropic)	Development assistance under student direction in the later phase: code review against the case study, pipeline, retrieval, entity-resolution, graph and UI implementation, tests, evidence capture, evaluation runs and updating this report and the submission package.	Code, tests, migrations, scripts, documentation, evidence screenshots, report and synopsis.	Student to confirm (Y/N)
Groq / openai-gpt-oss-120b	Runtime candidate extraction, ambiguous entity-merge decisions and grounded answer drafting through LLMProvider; external API dependency.	Candidate facts and answer drafts (always validated; never trusted directly).	Y
Google Gemini / gemini-embedding-2	Runtime chunk and query embeddings through EmbeddingProvider; 768-dimensional output.	Vector representations for chunks and queries.	Y

11. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved case-study scope, and that I can explain every significant part of the implementation.

Signature:		**Date:**	**7th October 2026**
Name:	Atharv Rao	**PRN:**	123B1B024